



ATM

DSD first year project



JUNE 3, 2025

NIȚĂ MARIA-ȘTEFANIA, ONIGA DANA
PROJECT SUPERVISOR: Ing. Ioana Virna

Table of Contents

1.	Specifications	2
2.	Design	2
2.1	Black Box.....	2
2.2	Control and Execution Unit	3
2.2.1	Resources (breakdown of the Execution Unit).....	4
2.2.2	Block Diagram for first breakdown	9
2.2.3	State diagram of the Control Unit.....	10
2.2.4	Detailed diagram of the project	14
3	User manual	15
4	Technical justifications for the design.....	17
5	Future developments	18
6	References.....	19

ATM

1. Specifications

Design an ATM for withdrawing sums in EURO. It is assumed that the maximum amount that can be withdrawn at once is 1,000 euros. Initially, card identification is performed and the operation is chosen. At least 4 different card accounts will be supported, and at least 4 different operations will be implemented. The ATM has a cash dispenser that initially holds a certain amount (number of banknotes of different denominations). In the case of a cash withdrawal request, the amount is entered, the availability of the requested sum is checked, the types of banknotes issued are displayed, and the account is updated. Then the card, the amount, and optionally, a receipt is issued. The project will be carried out by 2 students.

2. Design

2.1 Black Box

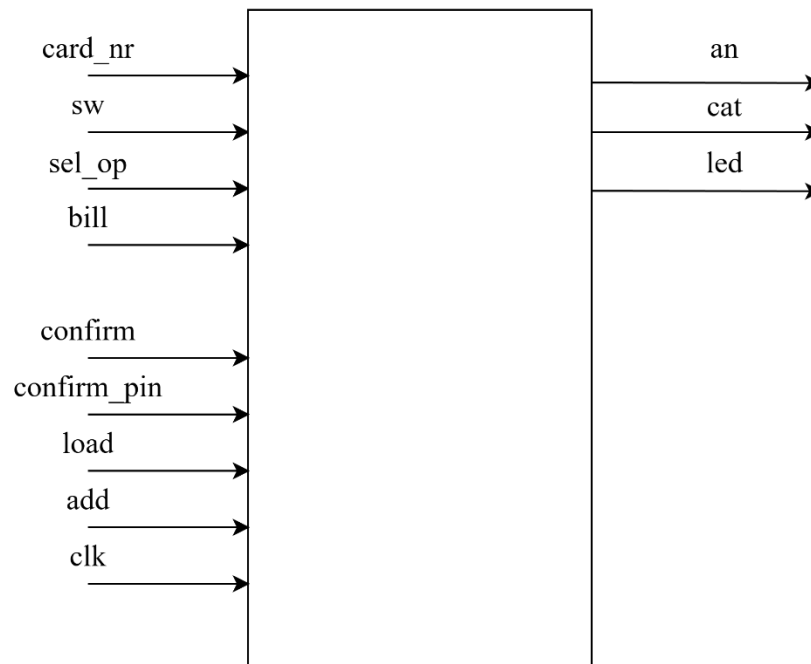


Figure 1 Black Box of the system

The initial implementation step in this ATM project requires careful attention, as some necessary inputs are not explicitly mentioned in the requirements, but are essential for the system to function on the Basys 3 FPGA board. For example, while the specification refers to actions like entering a PIN or confirming a withdrawal, it does not specify how these actions are triggered. In this case, we identified these missing actions and assigned them to buttons such as load, confirm, confirm_pin, and add.

2.2 Control and Execution Unit

The system's black box must be further broken down in order to find implementable components. We will do a **top-down** breakdown of the problem until we get to known circuits, and then we will implement **bottom-up**.

The first breakdown of any system is one in which we will differentiate between the **control logic** in the system and the **system resources**. The control logic is represented by the Control Unit (CU) and the resources are represented by the Execution Unit (EU). Any algorithm can be broken down in this way (*the abstract representation of an algorithm is done through a flow-chart*).

1. Mapping the inputs and outputs of the black box on the two components

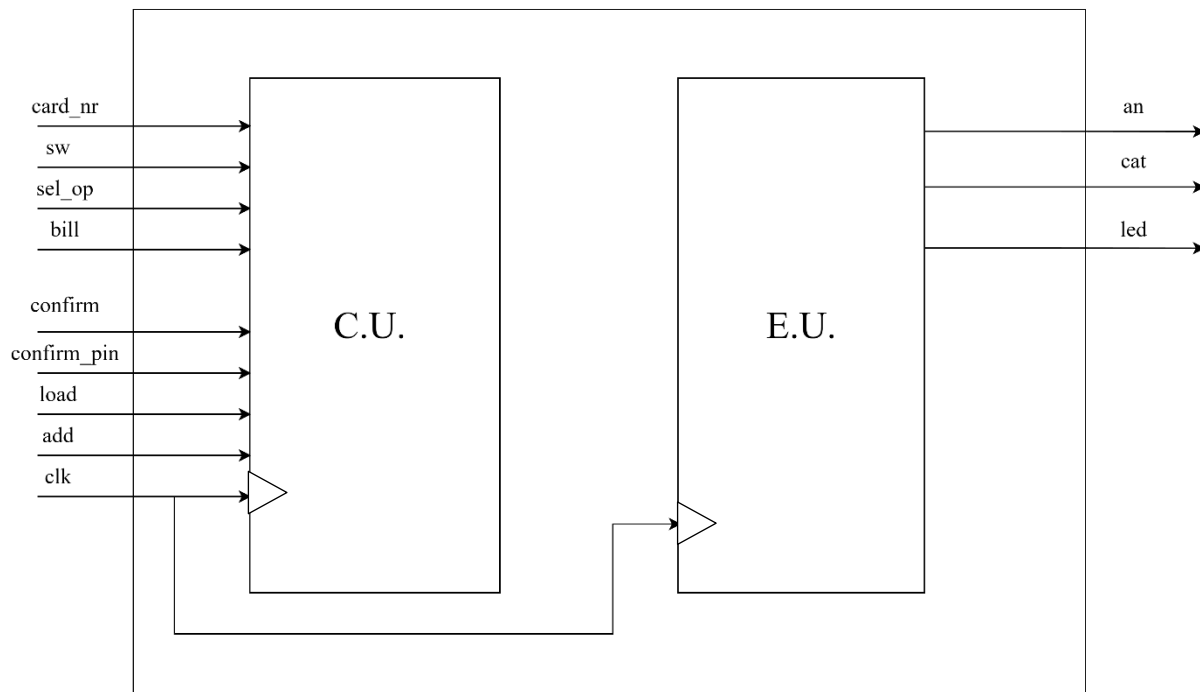


Figure 2 Mapping the inputs and outputs of the black box on the inputs and outputs of the units

We can divide both inputs and outputs into 2 categories: **data** and **control**. This separation is essential at the beginning.

Data inputs: card_nr [1:0], sw [3:0], sel_op [1:0], bill [2:0]

Control inputs: confirm button, confirm_pin button, load button, add button, clk

Data outputs: an [3:0], cat [6:0] used to print the output on the 7-segment display on Basys 3.

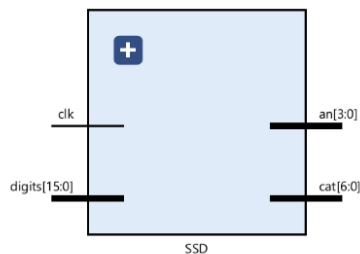
Control outputs: led, used to show if the inserted PIN is correct.

2. Resources (breakdown of the Execution Unit)

RESOURCES

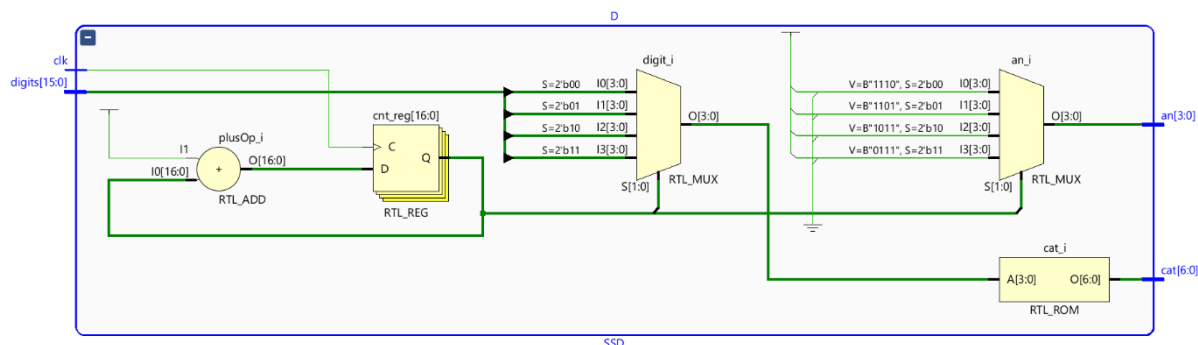
1. SSD:

This component is responsible for displaying 4-digit hexadecimal values on the 7-segment display of the Basys 3 board. It takes a 16-bit digit input and cycles through each 4-bit digit using multiplexing and a counter to control which digit is shown.



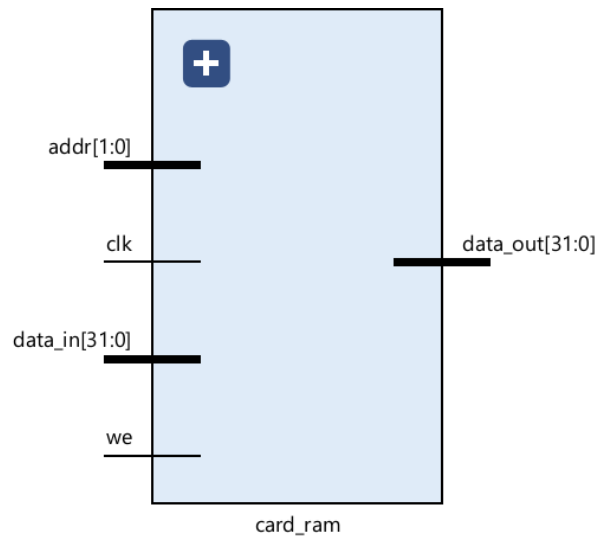
Internal components and their roles:

- **Counter:** Generates a refresh signal to cycle through each digit.
- **Multiplexer:** Chooses one of the four 4-bit segments from the 16-bit input to display on the current digit.
- **ROM:** Stores the 7-segment encodings for digits 0–F. Converts the binary digit into the appropriate output.



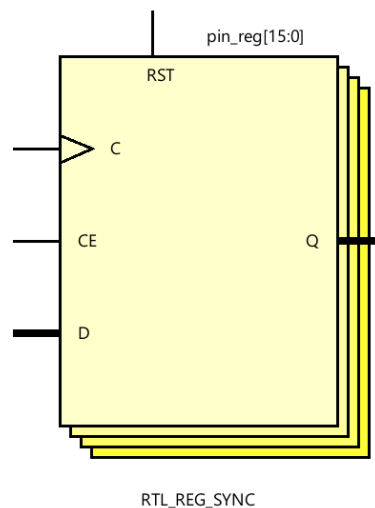
2. 32- bit RAM:

This component is used to store each card's PIN and balance. It supports read and write operations for 4 card accounts, selected using addr [1:0]. Data is written on the rising clock edge when 'we' is high and read continuously through data_out. This memory enables storing and updating user information during ATM operations.



3. 16-bit SHIFT REGISTER:

This component stores the 16-bit PIN entered via the switches. It updates on the rising clock edge when load_en is active, and each new 4-bit value is shifted in. It ensures the PIN is built up correctly before verification.

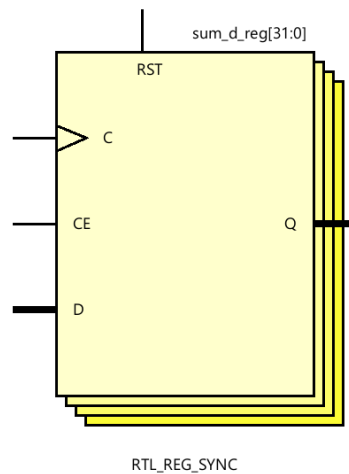


Other 16-bit SHIFT REGISTERS:

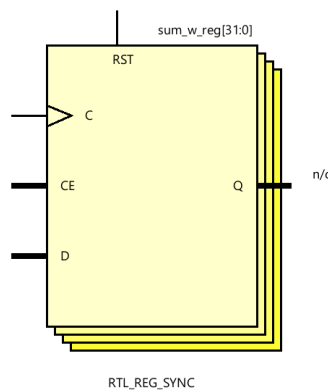
- pin_confirm: Stores the confirmed PIN for comparison. Updated when confirm_pin_en is active.
- pin_display: Temporarily holds the PIN for display.
- new_pin: Stores a new PIN when changing it.

4. 32-bit REGISTER:

This register stores the current deposit sum (sum_d). It updates on the rising edge of the clock and holds the value until a new one is written. It's used to accumulate the total deposit during a deposit operation.



Other 32-bit synchronous register stores the current withdrawal sum (sum_w). It updates on the rising edge of the clock when enabled and holds the value until the next valid change. It ensures the withdrawal amount is correctly tracked during the operation.



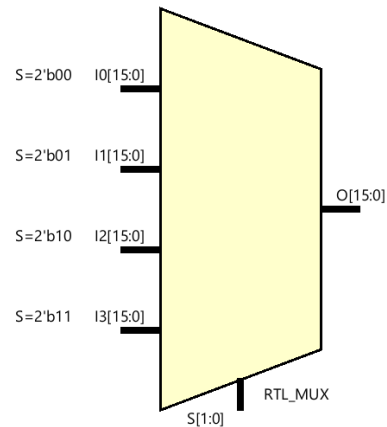
5. MUX 4:1

This component is used to select which 16-bit value is shown on the 7-segment display based on the selected ATM operation. It takes four 16-bit input values (I0 to I3) and outputs one of them to O [15:0] depending on the select signal S [1:0], which comes from sel_op. Each input corresponds to a possible operation:

- I0: SOLD
- I1: Change PIN
- I2: Deposit
- I3: Withdraw

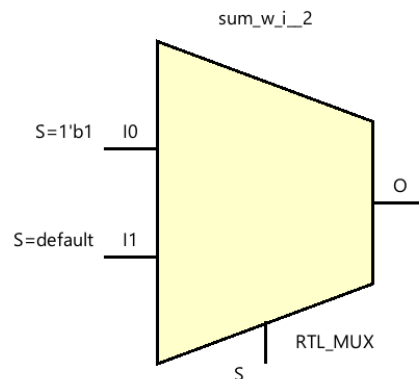
Other MUX 4:1 used:

- to control what gets shown on the display in case of error or success, selecting between standard outputs and predefined error codes
- select what data to write into RAM (ram_in) based on the operation. Inputs:
 - I0: hold value
 - I1: new PIN & old balance,
 - I2: old PIN & new deposit,
 - I3: old PIN & new withdrawal.



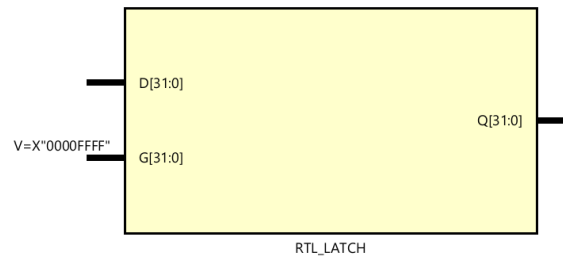
6. MUX 2:1

This component selects which value is assigned to the sum_w register during a withdrawal. It switches between keeping the value or updating it based on user input.



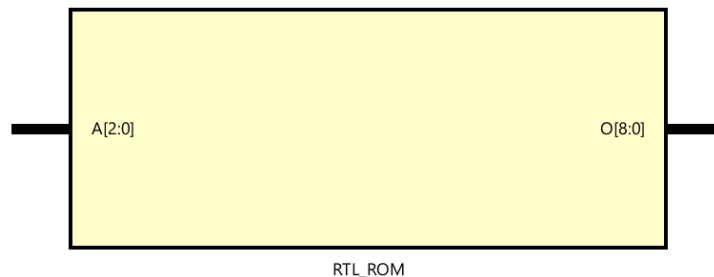
7. LATCH:

This component holds the updated balance after a successful withdrawal. It receives the new value D [31:0] and updates its output Q [31:0] when enabled. The constant G = x"0000FFFF" acts as a control signal to gate the update.



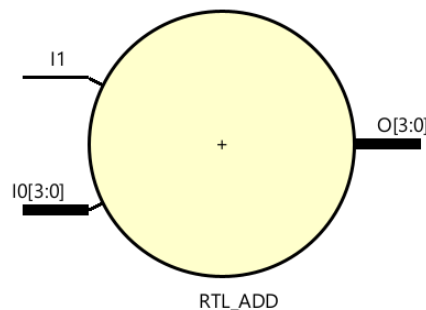
8. ROM:

This component takes a 3-bit bill input and outputs a 9-bit encoded value. Each code represents the value of a specific banknote (e.g., 5, 10, 20 EUR). It functions as a lookup table during deposit operations to update the total deposit (sum_d).



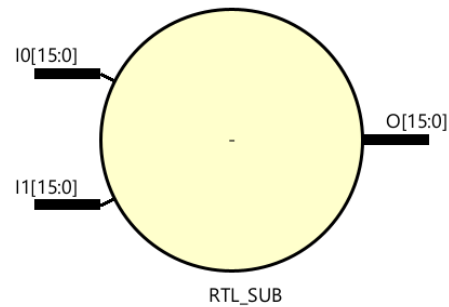
9. ADDER:

Performs 32-bit addition, used for updating the balance during deposit operations (sum_d + old_balance).



10. SUBTRACTOR:

Performs 16-bit subtraction, used during withdrawals to compute $\text{new_balance} = \text{old_balance} - \text{sum_w}$.



3. Block Diagram for first breakdown

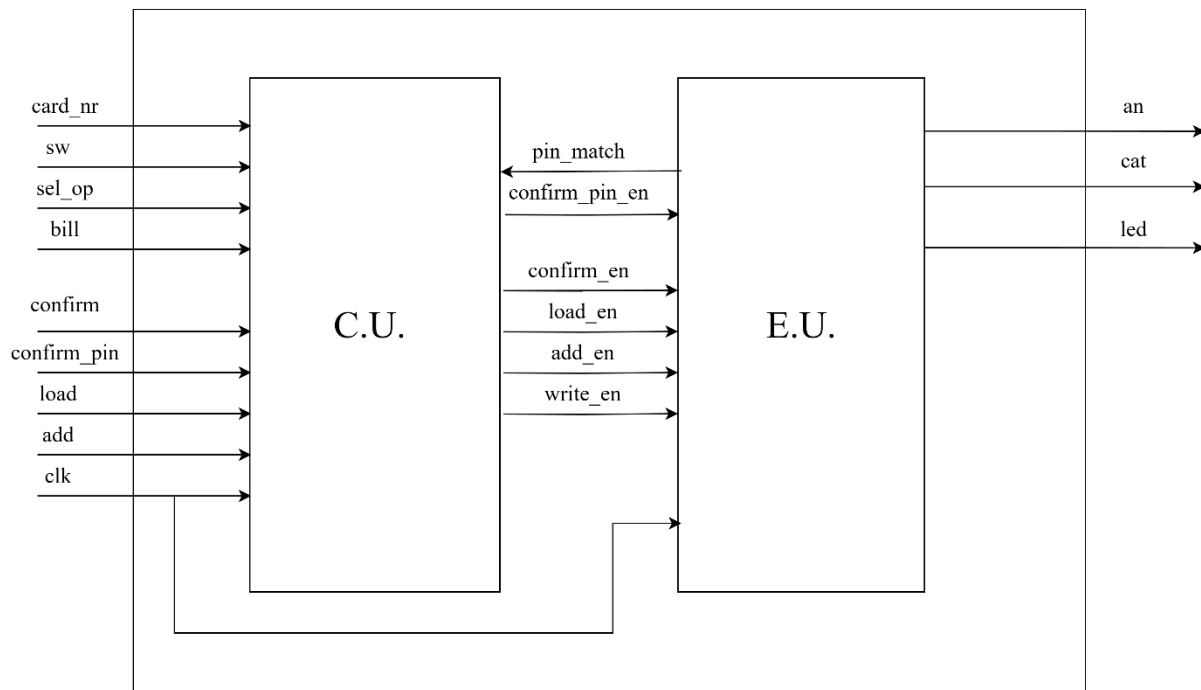
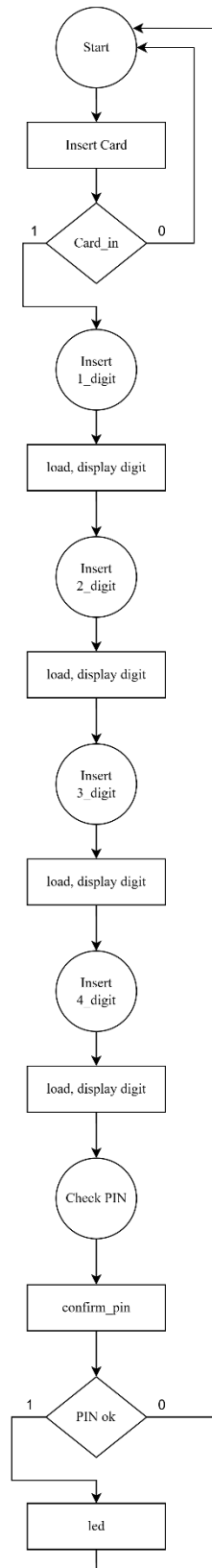
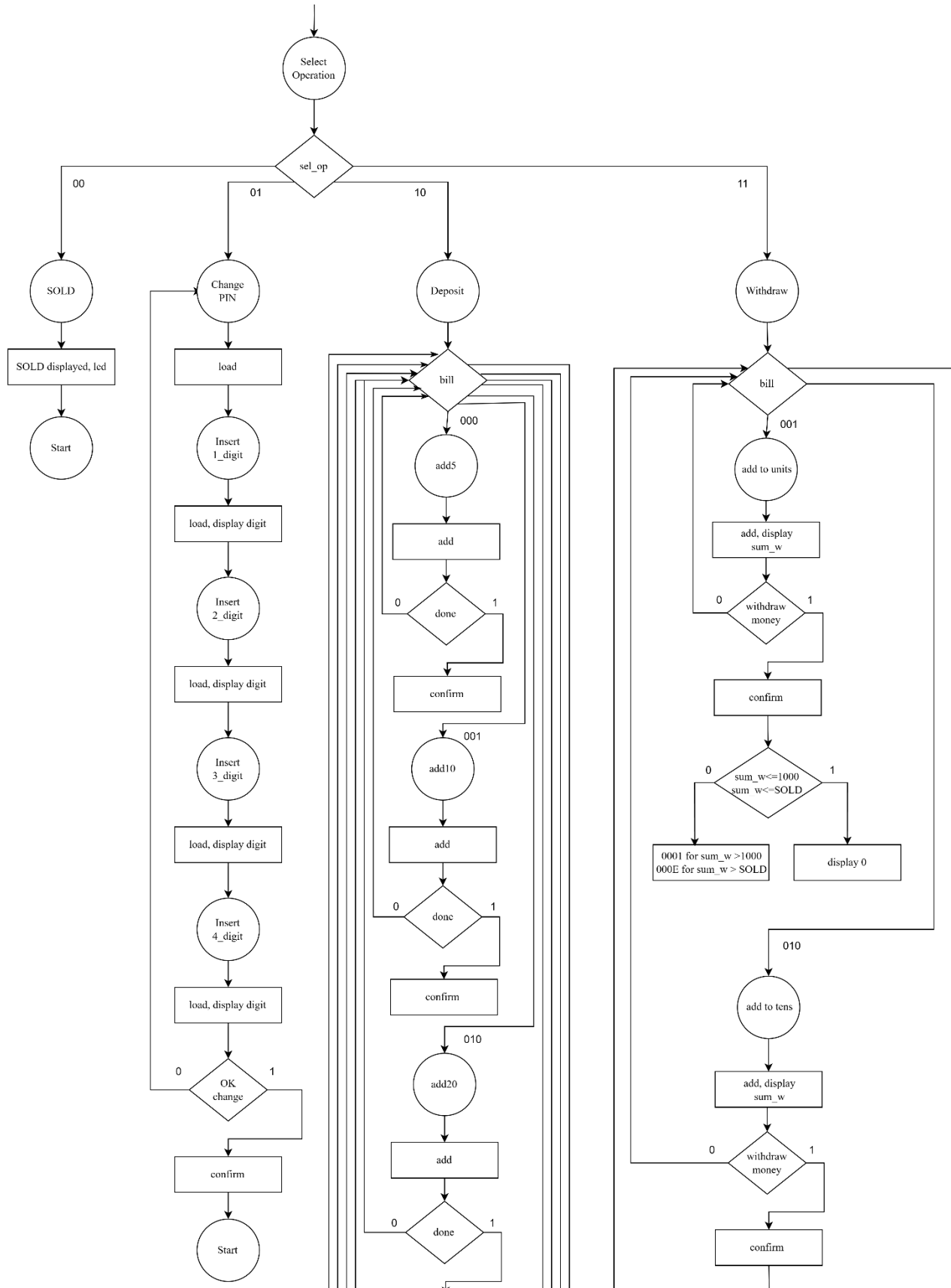
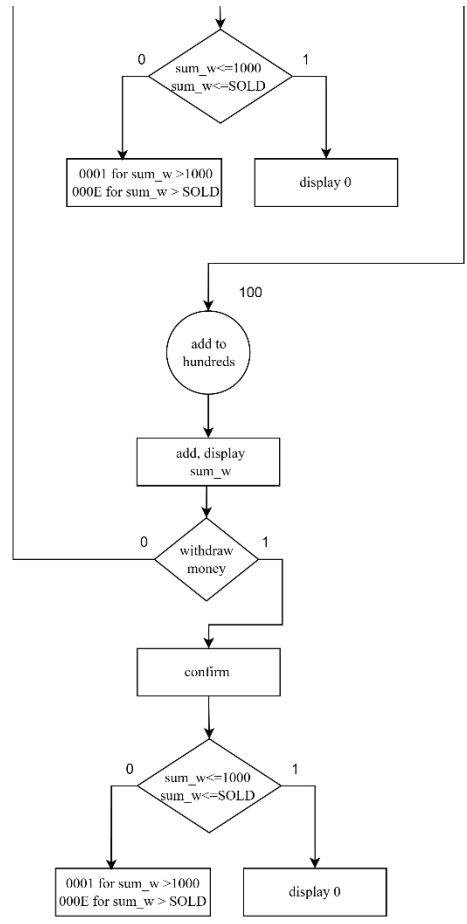
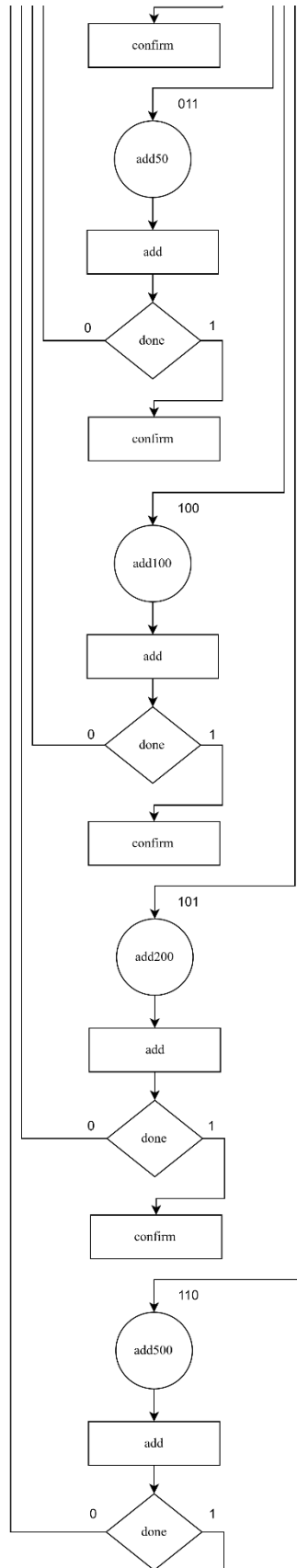


Figure 3 Mapping the connections found between the control and execution units

4. State diagram of the Control Unit







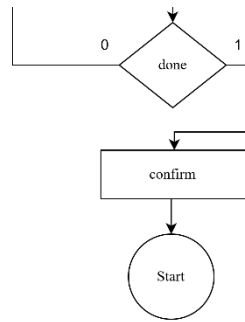
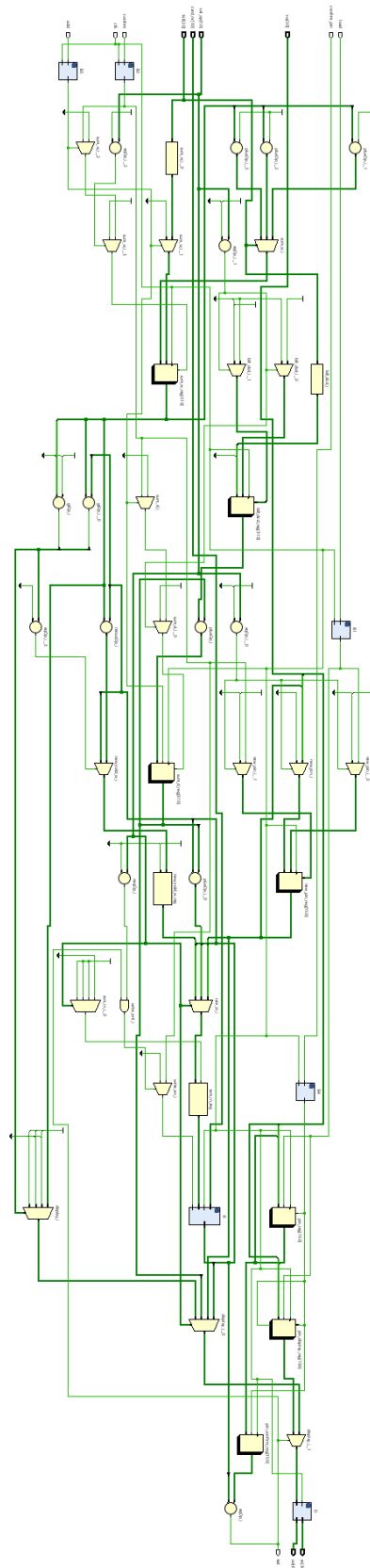
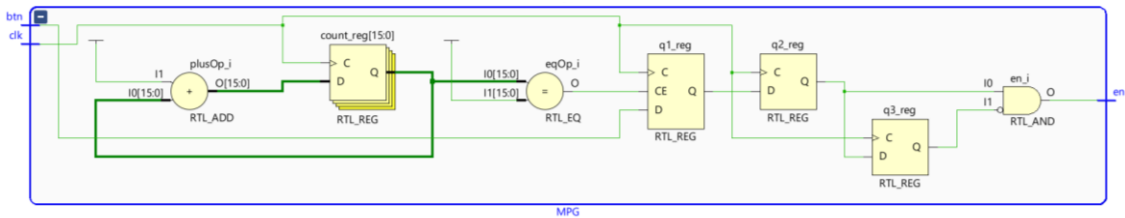


Figure 4 State diagram

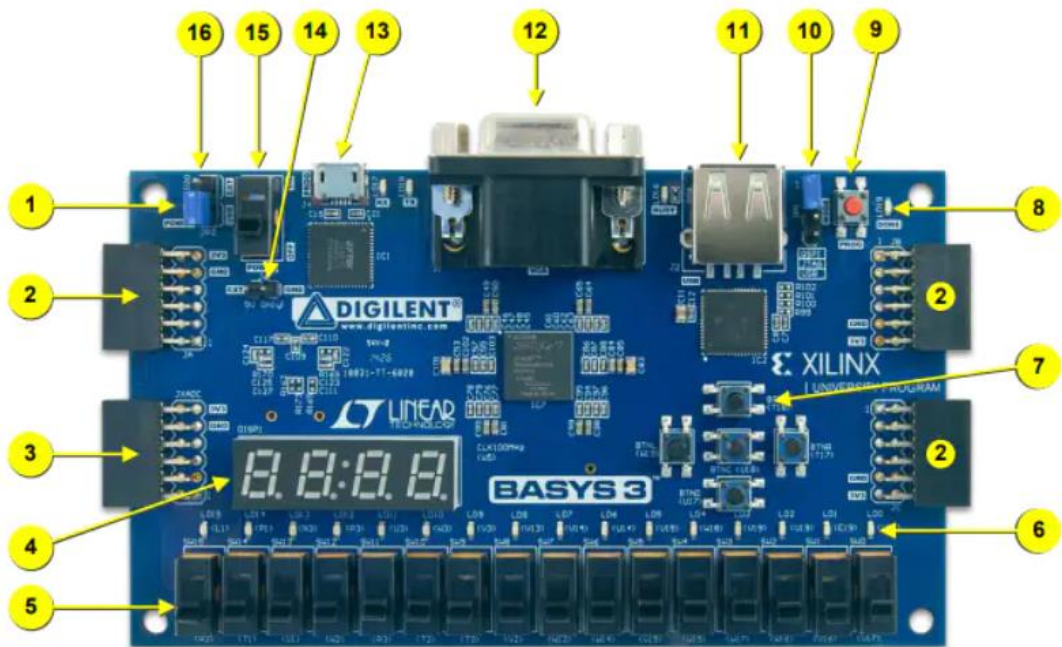
5. Detailed diagram of the project



MPG



3. User manual



Basys 3 FPGA board with callouts.

Callout	Component Description	Callout	Component Description
1	Power good LED	9	FPGA configuration reset button
2	Pmod port(s)	10	Programming mode jumper
3	Analog signal Pmod port (XADC)	11	USB host connector
4	Four digit 7-segment display	12	VGA connector
5	Slide switches (16)	13	Shared UART/ JTAG USB port
6	LEDs (16)	14	External power connector
7	Pushbuttons (5)	15	Power Switch
8	FPGA programming done LED	16	Power Select Jumper

The user begins by selecting the card they wish to access using the **card_nr** input, which is connected to W15 and V15 slide switches on the Basys 3 FPGA board. This action corresponds to choosing one of the four available card slots (card 0 to card 3). The ATM then enters the Insert PIN state, where the user is prompted to enter their 4-digit PIN code. The digits are entered one by one using the four input switches **sw** (V17, V16, W16, W17) with each digit being loaded into the system by pressing the **load** button (W19) after every entry. After all, four digits have been entered using this process, the user presses the **confirm_pin** button (U18) to submit the PIN. If the entered PIN matches the one stored in the internal RAM for the selected card, the ATM transitions to the Select Operation state, and the confirmation **led** (U16) lights up.

In the Select Operation state, the user chooses which action to perform by setting the **sel_op** switches (W14, W13). These two switches define one of four operations: 00 for SOLD, 01 for changing the PIN, 10 for depositing money and 11 for withdrawing money. Once the operation is selected, the user presses the **confirm** button (T18) to proceed.

SOLD: If this operation ($\text{sel_op} = 00$) is selected, the system immediately displays the current balance of the selected card on the four-digit seven-segment display. This amount is read from the internal card RAM and shown without the need for any additional button presses.

Change PIN: If this operation ($\text{sel_op} = 01$) is selected, the user inputs the new 4-digit PIN in the same way as the original PIN, digit by digit using the same switches, pressing the **load** button (W19) after each digit. After all four digits are entered, pressing the **confirm** button (T18) saves the new PIN into the selected card's memory. The system then returns to the idle state. The new PIN is immediately active, and the old one becomes invalid.

Deposit: If this operation ($\text{sel_op} = 10$) is selected, the user can add funds to the account by selecting the desired bill using the **bill** switches (U1, T1, R2). Each combination of these three switches corresponds to a specific bill value: 000 for €5, 001 for €10, 010 for €20, 011 for €50, 100 for €100, 101 for €200, and 110 for €500. For each bill to be deposited, the user sets the appropriate **bill** value and presses the **add** button (U17) once. This process is repeated for every bill they wish to deposit. The total deposit sum is accumulated internally and displayed on the 7-segment display. Once all bills have been entered, the user presses the **confirm** button (T18), which adds the deposit to the account balance and stores the updated value in RAM.

Withdraw: If this operation ($\text{sel_op} = 11$) is selected, the user builds the desired withdrawal amount in a digit-by-digit manner using the **bill** switches (U1, T1, R2). However, unlike the deposit operation, each **bill** value here selects a digit group rather than a fixed amount. The combination 100 increments the hundreds digit by 256 per press, 010 increments the tens digit by 16 per press, and 001 increments the units digit by 1 per press. The user presses the **add** button (U17) repeatedly to increment the desired digit group and construct the full withdrawal amount. The current amount is displayed on the 7-segment display. Once the desired amount is built (and is a multiple of 5), the user presses the **confirm** button (T18) to finalize the withdrawal. If the amount exceeds the maximum limit of 1000, or the current account balance, it is displayed on the 7-segment display '0001' respectively '000e'. Otherwise, the amount is deducted from the account, stored back into memory, and displays '0000'.

4. Technical justifications for the design

Registers and RAM:

In this project, synchronous registers were preferred for managing values like sum_d, sum_w, and pin, because they offer simpler implementation and faster access compared to RAM blocks. While RAM was used specifically for storing card data (PIN and balance), temporary and frequently updated values were managed via registers. This approach allowed easier control logic, better timing, and reduced design complexity, particularly for clocked updates.

ROM:

Instead of using combinational logic for decoding bill values, ROM blocks were used as lookup tables. This choice simplified the mapping between 3-bit bill types and their monetary values, enabling an organized way to define fixed constants like 5, 10, 20, 50, 100, 200, 500. The ROM design provided fast access to fixed data values while keeping the logic simple and efficient.

Integer Arithmetic:

The project uses integer arithmetic (addition and subtraction) for all balance updates. This decision fits ATM operations, which only deal with whole numbers like bills and balances. Using integers keeps the design simple and efficient, avoids the need for complex floating-point circuits. It also ensures faster and more reliable updates during deposits and withdrawals.

Multiplexers:

They were used throughout the project to control data routing efficiently. One MUX determines which value is displayed. Another MUX selects the input for the RAM depending on whether the user is viewing their account, changing the PIN, depositing funds, or withdrawing money. Other manage error messages and guide intermediate data paths. This approach ensures organized and flexible output control that adapts dynamically to user actions and system states.

Shift Register for PIN Input:

To allow entry of a 4-digit PIN using only a 4-bit switch input, the system uses a shift register mechanism. Each time the user presses the load button, 4 new bits are shifted into place, building the full 16-bit PIN step by step. This method allows accurate PIN entry while conserving input hardware, keeping the design compact and structured.

SSD:

The SSD module is responsible for showing values on the 7-segment display. It uses a counter to cycle through each digit position, a MUX is used to select one of the four 4-bit segments from the 16-bit input value, and a ROM to translate binary values into the correct 7-segment codes. Together, these components allow the display to update continuously and accurately, providing clear and real-time feedback to the user.

User Input:

Given the limited input options available on the Basys 3 board, user interaction was implemented using switches and buttons. To ensure reliable and noise-free inputs, each button was connected through an MPG (Mono Pulse Generator) module, which debounces the signal and produces a single, clean pulse for every press. This design allows precise control for critical actions such as PIN entry, value loading, and operation confirmation.

Conclusion:

The design choices in this ATM project are well-justified in terms of simplicity, efficiency, and hardware constraints. Registers and shift logic manage temporary values efficiently, ROMs handle decoding elegantly, and multiplexers streamline control. Integer operations match the application's financial nature, while SSD logic and user input mechanisms provide clear feedback and usability. Collectively, these decisions contribute to a balanced, practical, and resource-conscious FPGA design.

5. Future developments

While the current FPGA-based ATM system includes core functionalities such as withdrawal, deposit, and PIN change, several enhancements can be considered to expand its usability, security, and convenience. The following future developments aim to make the ATM system more secure, user-friendly, and versatile in a modern banking environment:

Biometric Authentication Instead of PIN:

To improve the security and ease of user authentication, a practical future development would be to replace the traditional 4-digit PIN system with biometric authentication. Users could authenticate themselves using fingerprint or face scanning technologies, ensuring that access to their accounts is tightly bound to their unique biological traits. This change would significantly reduce the risk of fraud due to stolen or guessed PINs, while also simplifying the user experience.

Transfers Between Cards:

Introducing card-to-card transfers would greatly enhance the system's functionality by enabling users to move funds between their own accounts or to other users within the banking network. This feature would include options to select a recipient card, specify the transfer amount, and confirm the transaction, offering a more flexible and comprehensive banking experience.

Contactless Cards:

To improve user convenience and promote hygienic interaction, the ATM could be upgraded to support contactless technology via NFC. This would allow users to initiate transactions simply by tapping a compatible bank card, smartphone, or wearable device, making access quicker and reducing physical contact with the terminal.

Voice Command Support:

For improved accessibility, especially for visually impaired users, voice command functionality can be added. The ATM would include a microphone and a noise-filtering system to accept spoken commands for operations like check balance or withdraw money.

6. References

<https://digilent.com/reference/programmable-logic/basys-3/reference-manual>

https://github.com/Digilent/Basys-3-GPIO/blob/master/src/constraints/Basys3_Master.xdc

<https://www.atmmarketplace.com/blogs/what-does-the-future-of-the-atm-industry-look-like/>